



Logo Fast and Mudry - AI generated

RAPPORT FAST & MUDRY

Projet de POO

Axel Schneider

Helder Ribeiro

10 juin 2026

Table des matières

1 – Introduction 3

1.1 – Défi et problématique 3

Note : Le rapport a été générée avec l'assistance d'une intelligence artificielle. Le contenu a été revu et validé par l'auteur avant son intégration au document final.

1 – Introduction

Dans le cadre du projet de programmation orientée objet, nous avons développé Fast & Mudry, un jeu de course réalisé en Scala avec la bibliothèque LibGDX. Le joueur contrôle un véhicule évoluant sur un circuit généré dynamiquement, tout en devant éviter différents obstacles et gérer les contraintes liées à la conduite.

Au cours du développement, plusieurs choix techniques ont été réalisés afin d'améliorer l'organisation du code et le rendu visuel du jeu. L'une des évolutions les plus importantes a notamment été le remplacement d'un système de rendu basé sur une perspective calculée directement dans le code par une approche reposant sur le Mode 7 et l'utilisation de shaders graphiques. Cette évolution a conduit à une réorganisation de certains composants du moteur de rendu tout en conservant la logique métier du jeu.

Ce rapport présente l'architecture logicielle du projet, les principaux mécanismes mis en œuvre ainsi que les choix de conception retenus. Une attention particulière est portée à l'organisation des différents modules, aux interactions entre les composants et à l'application des concepts de programmation orientée objet.

1.1 – Défi et problématique

1.1.1 – Rendu graphique

L'un des principaux défis rencontrés durant le développement de Fast & Mudry concernait le rendu graphique de la piste. Les premières versions du projet utilisaient un affichage de type arcade reposant sur une simulation de perspective calculée directement dans le code. Cette approche permettait de représenter une route et ses virages avec un coût de développement relativement faible, mais elle présentait plusieurs limitations en termes de lisibilité et de perception de l'environnement.

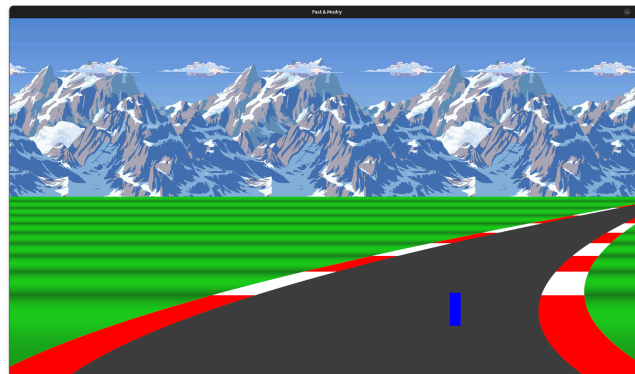


Figure 1 - Première version du rendu

En pratique, les virages étaient difficiles à anticiper pour le joueur. La représentation de la piste manquait de profondeur et les changements de direction apparaissaient souvent de manière abrupte. Cette limitation avait un impact direct sur le gameplay : le joueur disposait de peu d'informations sur ce qui se trouvait au-delà de la portion immédiatement visible de la route. Il devenait alors difficile d'anticiper les obstacles, de préparer une trajectoire adaptée ou d'évaluer la géométrie du circuit à moyen terme.

Afin d'améliorer cette situation, le système de rendu a été repensé autour d'une approche de type Mode 7. Cette technique consiste à projeter une texture représentant le circuit de manière à créer une illusion de profondeur tout en conservant un environnement essentiellement bidimensionnel. Contrairement à l'ancien rendu, cette approche offre une meilleure perception de l'espace et permet d'afficher une portion beaucoup plus importante du circuit devant le véhicule.



Figure 2 - Rendu avec mode 7

Cette évolution a considérablement amélioré la lisibilité du jeu. Le joueur peut désormais distinguer les virages à venir, observer les obstacles situés plus loin sur la piste et repérer certains éléments importants du décor, comme la HES qui matérialise la ligne d'arrivée. La visibilité accrue permet une conduite plus naturelle, fondée davantage sur l'anticipation que sur la réaction immédiate aux événements apparaissant à l'écran.

1.1.2 – Fonctionnement du rendu Mode 7 avec le GPU

Dans l'architecture actuelle, la responsabilité de définir la scène visuelle est déléguée à TrackRenderer. Cet objet agit comme un orchestrateur de rendu : il récupère la texture du circuit depuis le monde, positionne la caméra virtuelle à partir de l'état du véhicule, transmet ensuite un ensemble de paramètres au shader graphique et déclenche l'affichage. Cette séparation des responsabilités est cohérente avec un design modulaire, car la logique de jeu continue d'évoluer dans les classes métier, tandis que le rendu se limite à la transformation de l'état du monde en primitives graphiques exploitables par le GPU.

Le composant Mode7.scala ne calcule pas lui-même l'image finale. Il sert principalement de conteneur de configuration, en exposant les chemins du shader, les noms des uniforms et les valeurs par défaut utilisées par le moteur de rendu. Les uniforms sont ensuite envoyés depuis TrackRenderer vers le shader via le renderer graphique. Le shader, quant à lui, reçoit le contexte de rendu sous forme de variables globales, dont la résolution, la position de la caméra, son angle, l'axe de rotation, la distance du plan d'écran, l'inclinaison du point de vue et les facteurs de mise à l'échelle associés à la texture. Cette organisation permet de garder la logique de projection dans un composant spécialisé, indépendamment de la physique de conduite ou de la gestion de l'état du jeu. La caméra virtuelle est ainsi représentée par un ensemble de paramètres de projection et de position, ce qui rend le système facilement configurable et modulable.

La position et l'orientation de la voiture influencent directement le rendu. À chaque image, TrackRenderer met à jour la position de la caméra à partir des coordonnées du véhicule et son angle à partir de la rotation de la voiture. Ces valeurs sont transférées au shader sous forme d'uniforms, ce qui permet au GPU de recalculer la projection du circuit pour chaque pixel. Ainsi, lorsque le véhicule tourne, la texture du circuit est réinterprétée selon une nouvelle direction de regard, ce qui donne l'illusion d'un déplacement sur une route en perspective. Le système repose donc sur l'idée que l'état du véhicule est une entrée de rendu, au même titre que les dimensions de l'écran ou la résolution de la texture.

```

1 cameraPosition.y = World.INSTANCE.CAR.Coordinates.y
2 cameraPosition.x = World.INSTANCE.CAR.Coordinates.x
3 cameraAngle = World.INSTANCE.CAR.Rotation
4
5 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.POSITION, cameraPosition)
6 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.ANGLE, cameraAngle)

```

Listing 1 - Transmission des informations de déplacement au shader

Dans le shader, la transformation s'effectue à partir des coordonnées du fragment, c'est-à-dire des pixels de l'écran. Pour chaque pixel, le shader construit un rayon de vue dans un espace 3D synthétique, puis applique une rotation autour d'un axe défini par la caméra. Cette opération permet de transformer un point de l'écran en une direction de projection, puis d'inverser cette projection pour retrouver une position dans la texture du circuit. Le calcul combine ensuite la position de la caméra, la distance du plan d'écran et le facteur de rendu pour obtenir une coordonnée de texture comprise entre 0 et 1. Lorsque cette coordonnée appartient à la zone du circuit, la couleur correspondante est échantillonnée dans la texture et affichée à l'écran. Le mécanisme est donc fondamentalement un problème de géométrie projective, résolu en parallèle par le GPU pour chaque pixel.

```

1  vec3 pixelRay;
2  pixelRay.xz = (gl_FragCoord.xy / resolution.xy) * vec2(1.0, -1.0) + vec2(-0.5,
3  0.5);
4  pixelRay.y = screenPlanDistance;
5  pixelRay *= rotationMatrix(cameraAxis, cameraAngle);
6
7  vec2 hitPosition = cameraPosition.xy + pixelRay.xy * (cameraPosition.z /
8  pixelRay.z) + vec2(0.5, 0);
9  hitPosition = (hitPosition - mapOrigin) * renderingFactor;
10 gl_FragColor = texture2D(backbuffer, hitPosition);

```

Listing 2 - Projection du fragment vers une coordonnée de texture

Le GPU est particulièrement adapté à ce type de traitement car il exécute des opérations de calcul sur des millions de fragments de manière massivement parallèle. Contrairement à un calcul séquentiel dans le code CPU, le shader traite simultanément l'ensemble des pixels qui composent l'image. Cette parallélisation est particulièrement pertinente pour un rendu Mode 7, puisque chaque pixel doit être évalué indépendamment à partir de la même logique de projection. L'architecture adoptée est ainsi conforme aux principes de calcul graphique moderne, où le traitement géométrique et texturisé est délégué au processeur graphique afin de libérer le CPU pour la logique du jeu.

Le rendu repose sur une projection de la texture du circuit sur une surface virtuelle. Cette représentation reste simple et efficace pour un jeu de course 2D/2.5D, mais elle dépend fortement de la résolution de la texture, du réglage des paramètres de caméra et de l'alignement géométrique entre la piste et ses éléments décoratifs. Le shader doit également être paramétré avec précision afin d'éviter les artefacts aux limites du circuit ou dans les zones où la projection devient instable. Ce compromis est cohérent avec les objectifs du projet : offrir un rendu fluide et lisible sans introduire une complexité de calcul disproportionnée.

Le flux de données peut être résumé de la manière suivante : état du véhicule → uniforms transmis par TrackRenderer → shader GPU → échantillonnage dans la texture du circuit → image finale affichée à l'écran.